

GSoC 2026 Proposal

AI/ML Models for Agricultural Analytics on the National Research Platform

Predictive Irrigation, Plant Health Computer Vision, and Harvest Timing for the VINE Precision Agriculture Platform

Sohan Shingade

sshingade@ucsd.edu | UC San Diego | B.S. Data Science

Mentor: Mohammad Firas Sada (UCSD)

Project: VINE / AI/ML Models for Agricultural Analytics on NRP

Size: Large (350 hours) | Difficulty: Hard

1. Overview

Field	Details
Proposal Title	AI/ML Models for Agricultural Analytics on the National Research Platform
Project Mentor	Mohammad Firas Sada (UCSD)
Contributor	Sohan Shingade
Slack Handle	U0AM9S8GAQH / https://ucosre.slack.com/team/U0AM9S8GAQH
GitHub Handle	sohan-shingade
Email	sshingade@ucsd.edu
Timezone	Pacific Time (PT)
Size	Large - 350 hours

2. Introduction

2.1 Problem Statement

Agriculture in California faces compounding resource constraints: water scarcity, climate variability, labor shortages, and rising input costs. Vineyards and farms generate increasing volumes of sensor and imagery data, but most operational decisions - when to irrigate, which blocks need attention, when to harvest - still rely on manual observation and grower intuition. The data exists; what's missing are intelligent models that can turn raw sensor streams and drone imagery into actionable, timely predictions.

The VINE (Vineyard Intelligence Network & Environment) project addresses this gap. Iron Horse Vineyards in Sebastopol, California serves as a living laboratory where CENIC, UC San Diego, and partners have deployed a comprehensive sensor network and drone imaging capability. The raw agricultural data is being collected continuously - soil moisture, temperature, CO2 levels, weather conditions via LoRaWAN, plus periodic multispectral drone flights. This subproject focuses on building the ML models that transform this data into predictions: forecasting soil moisture to schedule irrigation proactively, analyzing multispectral imagery to detect plant stress and pest damage spatially, and predicting optimal harvest windows to optimize labor and logistics.

2.2 Background & Current State

Iron Horse Vineyards is the demonstration site - a working vineyard in Sonoma County where the full sensor infrastructure is already deployed and generating data. The vineyard has shown potential for a 10% reduction in water use through data-driven irrigation, demonstrating the practical value of precision agriculture.

Sensor data: Soil moisture, temperature, CO2 levels, and weather conditions are collected via LoRaWAN (low-power, wide-area network) sensors at regular intervals. LoRaWAN enables long-range, low-power communication ideal for distributed agricultural sensors across a vineyard. This data arrives as time-series streams indexed by sensor location and timestamp.

Multispectral drone imagery: Drones periodically capture images in multiple spectral bands: visible RGB, near-infrared (NIR), and red-edge. From these bands, vegetation indices can be computed that quantify plant health. The most common is NDVI (Normalized Difference Vegetation Index), calculated

as $(\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red})$. Healthy, photosynthetically active plants reflect strongly in NIR and absorb red light, producing high NDVI values. Stressed or diseased plants show lower NDVI. NDRE (Normalized Difference Red-Edge) provides additional sensitivity to chlorophyll content. These indices transform raw imagery into quantitative plant health metrics at spatial resolution.

Historical records: Past harvest dates, yields, irrigation schedules, and weather archives provide the supervised learning targets. Multi-year records enable models to learn seasonal patterns and inter-annual variability.

National Research Platform (NRP): All compute for this project runs on NRP's Nautilus Kubernetes cluster. NRP provides GPU pods (A100, L40, RTX A6000) for model training, persistent storage via Ceph, JupyterHub for interactive development, and a managed LLM service. The cluster is accessible to UCSD researchers and GSoC contributors with appropriate namespace access.

Current state: The sensor network and drone imaging capability are operational. A digital twin of the vineyard is being developed (separate VINE subproject). The AI/ML model layer - this subproject - is the component that turns raw sensor data and imagery into predictive insights. The data pipeline may be partially built or in early stages; exact status will be confirmed with the mentor during community bonding.

2.3 Proposed Solution

I propose building three interconnected ML model tracks, all trained on NRP's GPU clusters and deployed as inference services:

1. Predictive irrigation scheduling (time-series): Ingest soil moisture, weather forecasts, and historical irrigation data. Train forecasting models - ARIMA and Prophet as classical baselines, then LSTM for capturing nonlinear dynamics - to predict when soil moisture will drop below optimal thresholds. This gives operators advance warning to irrigate proactively rather than reactively, reducing both water waste and crop stress.
2. Plant health and pest detection (computer vision): Process multispectral drone imagery to compute vegetation indices (NDVI, NDRE), then train CNN-based classifiers (fine-tuned ResNet or EfficientNet) to detect plant stress, identify pest damage, and estimate yield from aerial imagery. This produces spatial health maps that show operators exactly which vineyard blocks need attention.
3. Harvest timing prediction (time-series + tabular): Combine sensor data, weather forecasts, cumulative growing degree-days, and historical harvest records to predict optimal harvest windows per vineyard block. This helps with logistics - labor scheduling, equipment allocation, and market timing.

All models share a common data ingestion pipeline (D1) and are evaluated systematically against naive and rule-based baselines. The final deliverable is a set of Dockerized inference services deployed on NRP Kubernetes, accessible via REST API for integration with the VINE dashboard and digital twin.

3. Project Goals

3.1 Objectives

- Build and evaluate predictive models for irrigation scheduling that outperform rule-based baselines (fixed-schedule irrigation, simple soil moisture thresholds).
- Develop computer vision models for plant health assessment from multispectral drone imagery, producing spatial health maps with per-block stress and pest detection scores.
- Create time-series forecasting models for harvest timing prediction, providing per-block harvest readiness scores with confidence intervals.
- Train all models on NRP GPU clusters with reproducible experiment tracking (logged hyperparameters, loss curves, evaluation metrics).
- Deploy models as containerized inference services on NRP Kubernetes, accessible via REST API for integration with the VINE dashboard and digital twin.
- Document the full pipeline for reproducibility and future VINE contributors, with bi-weekly blog posts tracking progress.

Community benefit: Vineyard operators get actionable predictions - when to irrigate, which vines need attention, when to harvest. The open-source models, pipelines, and processed datasets benefit the broader agricultural research community and establish a reproducible framework for precision agriculture ML on NRP.

3.2 Expected Deliverables

#	Deliverable	Description
D1	Data Ingestion & Feature Pipeline	Python module to ingest sensor data + drone imagery + historical records, clean/validate, compute vegetation indices, and engineer features for all model tracks
D2	Irrigation Scheduling Models	Time-series forecasting models (ARIMA, Prophet, LSTM) predicting soil moisture at 6h/12h/24h/48h horizons with irrigation recommendation layer
D3	Plant Health CV Models	CNN-based models (fine-tuned ResNet/EfficientNet) analyzing multispectral imagery for plant stress classification, pest detection, and yield estimation
D4	Harvest Timing Models	Forecasting models (XGBoost, LSTM) predicting optimal harvest windows from sensor + weather + historical data
D5	Model Evaluation Report	Systematic comparison across all model tracks: metrics, ablation studies, visualizations, forecast-vs-actual plots, spatial health maps
D6	NRP Deployment	Dockerized models deployed as K8s inference services on NRP with REST API endpoints (FastAPI)
D7	Documentation & Blog Posts	README, API docs, bi-weekly status blog posts, final project summary

3.3 Future Work

- **Multi-site generalization:** Adapt models to other vineyards and farms beyond Iron Horse using transfer learning, testing how well models trained on one site generalize to different climates, soils, and grape varieties.

- **Digital twin integration:** Feed model predictions directly into the VINE digital twin (separate subproject) for interactive 3D visualization of predicted irrigation needs, health maps, and harvest readiness overlaid on the vineyard geometry.
- **Operator feedback loop:** Incorporate operator responses to irrigation recommendations (accepted/rejected/modified) to retrain models and calibrate sensitivity over time.
- **Multi-year climate analysis:** As additional seasons of data accumulate, retrain models to capture long-term climate adaptation trends and inter-annual variability in vine behavior.

4. Implementation Plan

4.1 Methodology

Implementation follows iterative 2-week phases, each producing a working, testable artifact. The three model tracks (irrigation, CV, harvest) share a common data pipeline (D1) and evaluation framework (D5), reducing duplicated effort.

- **Iterative delivery:** Each phase ends with a working component that can be independently tested and demonstrated. No big-bang integration at the end.
- **Communication:** Weekly 1:1 syncs with Mohammad Firas Sada. Bi-weekly blog posts on OSRE. Active on the OSRE Slack channel.
- **Version control:** Public GitLab repo (gitlab.nrp-nautilus.io) with CI from week 1. All experiments logged with hyperparameters and metrics.
- **Evaluation-driven:** Every model is compared against naive and rule-based baselines. Nothing ships without quantitative evidence it adds value over simpler approaches.
- **Priority ordering:** D2 (irrigation) first as the most data-rich and closest to pure time-series forecasting. D3 (CV) second as the highest-impact deliverable. D4 (harvest) third, scoped flexibly based on data availability.

4.2 Technical Architecture

4.2.1 Data Ingestion & Feature Pipeline (D1)

What I'm building: A Python module that ingests raw agricultural data from multiple heterogeneous sources and outputs clean, model-ready datasets for all three model tracks.

Data sources:

- Sensor time-series: Soil moisture, temperature, CO₂, weather from LoRaWAN sensors (likely JSON or CSV, timestamped, indexed by sensor location)
- Multispectral drone imagery: GeoTIFF or similar raster format with RGB + NIR + Red-edge bands, georeferenced to vineyard coordinates
- Historical records: Past irrigation schedules, harvest dates, yields, weather archives - provides supervised learning targets

What I need to implement:

- Sensor data reader: parse timestamps, handle missing readings from sensor failures or connectivity drops, resample to regular intervals (e.g., hourly)
- Image preprocessing: load multispectral bands with rasterio, compute vegetation indices ($NDVI = (NIR - Red) / (NIR + Red)$, $NDRE = (NIR - RedEdge) / (NIR + RedEdge)$), tile orthomosaics into patches for CNN input
- Geospatial alignment: register drone images to vineyard block coordinates using geopandas, enabling per-block aggregation of health scores
- Time-series feature engineering: rolling averages of soil moisture (1h/6h/24h), cumulative growing degree-days, lag features, weather forecast integration
- Data validation: range checks, outlier detection, completeness reports, staleness flags for gaps

Tools: Python, pandas, numpy, rasterio, rioxarray, geopandas, xarray

Key challenge: Sensor data will have gaps from sensor failures or connectivity issues. Drone imagery frequency depends on weather and flight schedules. The pipeline must handle both gracefully with robust imputation and quality flags.

4.2.2 Irrigation Scheduling Models (D2)

What I'm building: Time-series forecasting models that predict soil moisture levels at multiple horizons and recommend when to irrigate.

Approach:

- Target variable: Soil moisture at 6h, 12h, 24h, and 48h forecast horizons
- Features: Current soil moisture, temperature, humidity, recent rainfall, weather forecast (if available), time-of-day, day-of-year, cumulative evapotranspiration proxy

Classical baselines:

- ARIMA: Fit per-sensor models using pmdarima.auto_arima with stationarity testing (ADF) and automatic order selection
- Prophet: Per-sensor models with daily and weekly seasonality, temperature as external regressor, changepoint detection for irrigation events

Deep learning:

- LSTM encoder-decoder: Input = sliding window of past N hours of sensor readings + weather features. Encoder: 2-layer LSTM (hidden ~128). Decoder: linear projection to soil moisture predictions at each horizon. Multi-task head predicts soil moisture + temperature jointly.

Irrigation decision layer:

- When predicted soil moisture crosses a configurable threshold (set per vineyard block based on soil type and grape varietal), trigger an irrigation recommendation with confidence interval and lead time

Evaluation:

- Walk-forward validation on held-out time periods (train on spring, validate on summer)
- Metrics: MAE and RMSE on soil moisture predictions; precision/recall on irrigation trigger decisions
- Baselines to beat: fixed-schedule irrigation, simple threshold rule (irrigate when current moisture < X)

Tools: statsmodels, pmdarima, prophet, PyTorch

4.2.3 Plant Health CV Models (D3)

What I'm building: Computer vision models that analyze multispectral drone imagery to assess plant health, detect pest damage, and estimate yield.

Input processing:

- Load multispectral orthomosaics with rasterio, compute NDVI and NDRE as additional channels
- Tile into fixed-size patches (e.g., 256×256 pixels) aligned to vineyard block boundaries

- Each patch gets a feature vector: [R, G, B, NIR, RedEdge, NDVI, NDRE] - 7-channel input to the CNN

Task 1 - Plant stress classification:

- Binary or multi-class classification per patch: healthy / mild stress / severe stress
- Fine-tune a pretrained CNN (ResNet-50 or EfficientNet-B0) - replace final FC layer, add 7-channel input adapter, train with cross-entropy loss
- If labeled imagery is limited, start with NDVI thresholding as pseudo-labels for initial training, then refine with manual annotations

Task 2 - Pest/disease detection:

- Patch classification for visible pest damage patterns in imagery
- Data availability determines scope - discuss with mentor what labeled pest imagery exists. If insufficient, this task becomes anomaly detection against healthy baselines.

Task 3 - Yield estimation:

- Regression model predicting yield (e.g., tons/acre) per block from image features + sensor data
- Combine CNN features with tabular features (soil moisture history, GDD, historical yields) in a fusion model

Training:

- Data augmentation: random crops, flips, rotations, brightness/contrast jitter via albumentations
- Train on NRP GPU pods (A100 or L40)
- Experiment tracking: log all hyperparameters, loss curves, and per-class metrics

Evaluation:

- Accuracy, F1-score, confusion matrix for classification tasks
- MAE for yield regression
- Visual inspection: overlay predictions on drone imagery to produce spatial health maps

Tools: PyTorch, torchvision, OpenCV, rasterio, albumentations, geopandas

Key challenge: Labeled training data for plant stress and pest detection may be limited. Mitigation: start with unsupervised vegetation indices as pseudo-labels, use transfer learning from ImageNet, discuss data annotation strategy with mentor early.

4.2.4 Harvest Timing Models (D4)

What I'm building: Forecasting models that predict optimal harvest windows per vineyard block.

Approach:

- Target: Days-until-harvest regression, or binary harvest-ready classification
- Features: Cumulative growing degree-days (GDD), sugar content trends (Brix, if available from lab sampling), recent and forecasted weather, soil moisture trajectory, historical harvest dates for this block, NDVI trend from recent drone flights
- Tabular baseline: XGBoost with engineered features - simple, interpretable, handles mixed feature types well

- Sequential model: LSTM processing the full season's daily feature vector leading up to harvest, capturing temporal progression patterns
- Output: Per-block harvest readiness score (0–1) with confidence interval, plus estimated days-until-optimal-harvest

Evaluation:

- MAE on days-until-harvest prediction
- Precision/recall on harvest-ready classification (at various threshold cutoffs)
- Baseline: historical average harvest date for each block (predict same date as last year)

Tools: scikit-learn, XGBoost, PyTorch

Key challenge: Harvest happens once per year per block, so labeled data is inherently sparse. Multi-year historical records are critical. If insufficient years of data exist, this deliverable can be scoped to an exploratory analysis with the available data and framed as a prototype for future seasons.

4.2.5 Model Evaluation Report (D5)

What I'm building: A systematic comparison across all three model tracks, serving as both a technical report and the basis for the final blog post.

- Per-track comparison: naive baseline → classical model → deep learning, with quantitative metrics at each step
- Cross-track analysis: which data sources contribute most? Ablation studies removing weather features, removing imagery, removing historical data
- Visualizations: predicted vs. actual soil moisture time series, spatial health maps overlaid on vineyard imagery, harvest readiness timelines per block
- Failure analysis: where do models fail? Specific sensor types, specific vineyard blocks, specific weather conditions?

4.2.6 NRP Deployment (D6)

What I'm building: Containerized inference services deployed on NRP's Kubernetes cluster.

Architecture:

- Dockerfile per model track: irrigation-inference, cv-inference, harvest-inference
- K8s Deployments + Services exposing REST API endpoints via FastAPI
- GPU pod for batch inference on new drone imagery (triggered when new orthomosaic is uploaded)
- CronJob for scheduled sensor data re-ingestion and irrigation forecast refresh
- Integration point: API outputs are designed to feed into the VINE dashboard and digital twin (other subprojects)

API design:

- GET /irrigation/forecast?block_id=X → returns predicted soil moisture + irrigation recommendation
- POST /health/analyze (upload image patch) → returns stress classification + NDVI score
- GET /harvest/readiness?block_id=X → returns harvest readiness score + confidence interval

Tools: Docker, Kubernetes, FastAPI, kubectl

4.3 Challenges & Mitigations

Challenge	Mitigation
Limited labeled drone imagery for CV	Start with unsupervised NDVI thresholding as pseudo-labels; use transfer learning from ImageNet; discuss annotation strategy with mentor early
Sensor data gaps and noise	Robust imputation with staleness flags and quality scores; focus on sensors with best coverage first; distinguish sensor failure from real signal
Three model tracks = scope risk	Strict priority order: D2 (irrigation) first, D3 (CV) second, D4 (harvest) third. D4 is scoped flexibly based on data availability
Harvest data is inherently sparse (1x/year)	Use multi-year records; if insufficient, scope D4 as exploratory prototype for future seasons
Multispectral image processing complexity	Leverage existing geospatial experience (Wildfire project used rasterio/rioxarray); start with NDVI computation before full CNN pipeline
K8s deployment complexity	Start with Docker locally; graduate to K8s with mentor guidance; follow NRP's existing deployment patterns

5. Project Timeline

Total: 350 hours over 13 weeks (May 25 – August 24, 2026)

Note: Weeks 1–2 overlap with UCSD finals (through ~June 7). Work is kept light during this period.

Weeks	Dates	Phase	Deliverables	Hours
1–3	May 25 – Jun 14	Community Bonding	Setup NRP env, explore IHV datasets, understand sensor schema + drone imagery format, meet mentor (light W1–2 finals)	20
3–5	Jun 8 – Jun 28	Data Pipeline	D1: Ingestion pipeline for sensors + imagery + historical data, feature engineering, vegetation index computation, data validation	50
5–7	Jun 22 – Jul 12	Irrigation Models	D2: ARIMA/Prophet baselines + LSTM for soil moisture forecasting, irrigation decision layer, walk-forward evaluation	55
7–9	Jul 6 – Jul 26	CV Models	D3: Multispectral preprocessing, NDVI/NDRE computation, CNN fine-tuning for plant stress + pest detection, spatial health maps	70
-	~Jul 12	MIDTERM EVAL	D1, D2 complete. D3 in progress.	-
9–10	Jul 27 – Aug 9	Harvest + Eval	D4: Harvest timing models + D5: Cross-model evaluation report with ablation studies	50
11–12	Aug 3 – Aug 16	Deployment	D6: Dockerize all models, deploy as K8s inference services on NRP with FastAPI endpoints	50
12–13	Aug 10 – Aug 24	Docs + Polish	D7: Documentation, API docs, final blog post, code cleanup, handoff notes	55
Ongoing	-	Communication	Bi-weekly blog posts, Slack updates, weekly mentor syncs	(incl.)

Time conflicts:

UCSD spring quarter finals run through approximately June 7, 2026. During weeks 1–2, I will be available at reduced capacity (~10 hours/week) for community bonding activities only. Full-time availability begins week 3 (June 8).

6. Biographical Information

6.1 Relevant Experience

Academic:

I'm a third-year Data Science student at UC San Diego, graduating June 2027. My coursework is unusually well-aligned with this project's requirements, spanning both the computer vision and time-series tracks. I've taken Computer Vision (CNN architectures, image classification, object detection), Remote Sensing & Geospatial Analytics (working with satellite and aerial imagery, vegetation indices, raster processing), Time Series Analysis (ARIMA, stationarity, forecasting evaluation), Machine Learning, Deep Learning, and Statistical Learning. This combination of CV, geospatial, and time-series coursework directly maps to the three model tracks in this proposal.

Work Experience:

At Yonder Dynamics (ML/Perception Engineer, Sep 2024–present), I built real-time perception modules using PyTorch and OpenCV, designed end-to-end dataset pipelines with active learning, and implemented evaluation harnesses for model robustness across domain shift and lighting variation. The computer vision pipeline experience - from data collection through evaluation - transfers directly to the multispectral drone imagery analysis in D3.

At Shepherd Ventures (ML Engineer, Sep 2024–present), I built reproducible Python and SQL feature pipelines and model training workflows, developed experiment tracking and evaluation suites with standardized metrics, and containerized batch jobs with Docker. The emphasis on reproducibility, systematic evaluation, and containerization maps to the evaluation report (D5) and K8s deployment (D6) deliverables.

At the UCSD ECE Makerspace (Technical Lab Assistant, Sep 2024–present), I mentor 200+ students and author documentation and SOPs. This collaborative environment experience is relevant to the open-source documentation and community engagement expected in GSoC.

Projects:

Wildfire Ignition Risk Heatmap is my most directly relevant project for this proposal. I built an end-to-end geospatial ML pipeline processing gridMET netCDF climate data, LANDFIRE fuels rasters, census population density, and CAL FIRE burn perimeters. The tools I used - rasterio, rioxarray, geopandas, xarray - are the exact same tools needed for processing multispectral drone imagery in D3. The multimodal feature engineering from heterogeneous raster and vector data sources is directly analogous to combining drone imagery, sensor data, and historical records. I also built an interactive Folium heatmap for error analysis, similar to the spatial health maps this project will produce.

Power Outage Analysis & Forecasting demonstrates my time-series forecasting capability with messy real-world data. I built classification and regression models for predicting outage occurrence and restoration time, implementing walk-forward evaluation with statsmodels and scikit-learn. The project involved cleaning irregular records, running hypothesis tests, and producing uncertainty intervals - workflows I'll replicate for the irrigation and harvest timing models.

6.2 Technical Strengths

- **Core:** Python, PyTorch, scikit-learn, pandas, NumPy, statsmodels, SQL

- **Computer Vision:** OpenCV, torchvision, CNN architectures (ResNet, EfficientNet), image augmentation - from CV coursework + Yonder Dynamics
- **Geospatial:** geopandas, rasterio, rioarray, xarray, GDAL, Folium, QGIS - from Remote Sensing coursework + Wildfire project
- **Time Series:** ARIMA (stationarity, ACF/PACF, differencing), Prophet, LSTM - from Time Series Analysis coursework + Power Outage project
- **Infrastructure:** Docker, AWS (S3, EC2), Git, Jupyter; foundational Kubernetes knowledge through containerization experience

6.3 Why This Project

I'm a UCSD student, and NRP's infrastructure lives at SDSC on my campus. Mohammad Firas Sada is a UCSD researcher, which creates the opportunity for in-person collaboration and faster iteration - an advantage most remote GSoC contributors don't have.

What drew me to this specific VINE subproject is how precisely it aligns with my skill set. Most applicants will be strong in either computer vision or time-series forecasting, but this project requires both, plus geospatial data processing. My Remote Sensing & Geospatial Analytics coursework and Wildfire project gave me hands-on experience with rasterio, rioarray, and vegetation index computation, the exact tools needed for multispectral drone imagery analysis. My Time Series Analysis course and Power Outage project gave me the forecasting foundations for the irrigation and harvest models. And my Computer Vision coursework and Yonder Dynamics perception work prepared me for the CNN-based plant health classification. The intersection of these three areas is unusual, and it's exactly what VINE needs.

I'm also motivated by the tangible impact. California's water crisis makes precision irrigation a genuinely urgent problem. The project has already demonstrated potential for 10% water use reduction at Iron Horse. Extending that through better ML models, and deploying them as real services that vineyard operators actually use, is the kind of work that matters beyond a summer project.

Finally, I'm honest about what I want to learn. I have Docker experience but limited Kubernetes exposure, and deploying ML models as production inference services on one of the largest academic K8s clusters in the country is an opportunity I can't get from coursework alone.

6.4 Contact Information

Field	Value
Full Name	Sohan Shingade
Email	sshingade@ucsd.edu
Affiliation	UC San Diego, B.S. Data Science (expected June 2027)
Timezone	Pacific Time (PT)
GitHub	github.com/sohan-shingade
LinkedIn	https://www.linkedin.com/in/sohan-shingade/